

Experiment #10

Implementation of Matrix Multiplication Using CUDA

Objective:

Objective of this lab is to get the knowledge about the implementation of matrix multiplication in parallel using GPU.

Software Tools:

✓Ubuntu

Theory:

Introduction: -

CUDA is a parallel computing platform and programming language that allows software to use certain types of graphics processing unit (GPU) for general purpose processing, an approach called general-purpose computing on GPUs (GPGPU). It could significantly enhance the performance of programs that could be computed with massive parallelism.

Matrix multiplication is a typical application that could be computed with massive parallelism.

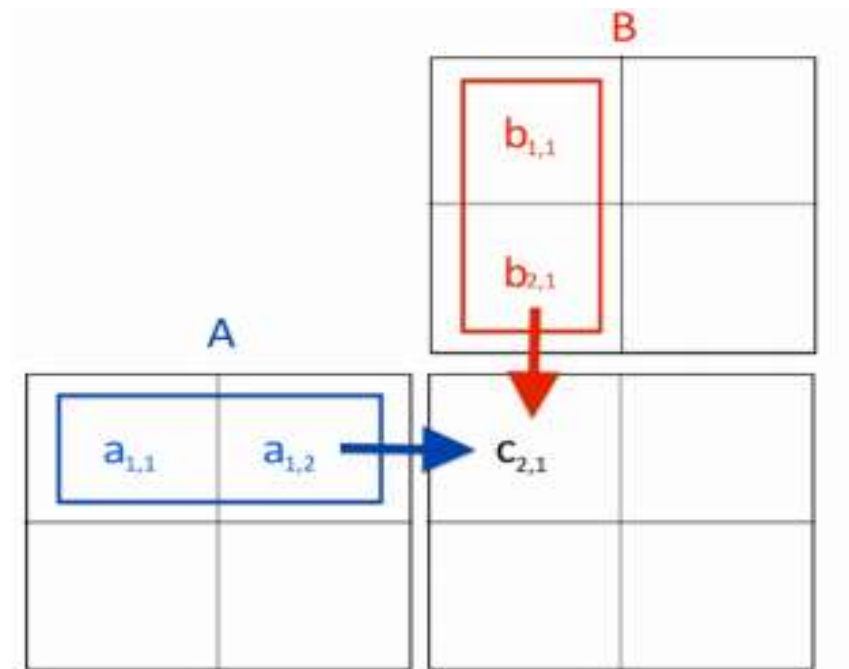
Matrix Multiplication:

$$\begin{aligned} \mathbf{C}^{n \times p} &= \mathbf{A}^{n \times m} \mathbf{B}^{m \times p} \\ \mathbf{C}^{b \times n \times p} &= \mathbf{A}^{b \times n \times m} \mathbf{B}^{b \times m \times p} \end{aligned}$$

Before starting, it is helpful to briefly recap how a matrix-matrix multiplication is computed. Let us say we have two matrices, A and B. Assume that A is a $n \times m$ matrix, which means that it has n rows and m columns. Also assume that B is a $m \times w$ matrix. The result of the multiplication $A \times B$ (which is different from $B \times A$!) is a $n \times w$ matrix, which we call M. That is, the number of rows in the resulting matrix equals the number of rows of the first matrix AA and the number of columns of the second matrix BB.

Why does this happen and how does it work? The answer is the same for both questions here. Let us take the cell 1,1 (first row, first column) of MM. The number inside it after the operation $M=A \times B$ is the sum of all the element-wise multiplications of the numbers in AA, row 1, with the numbers in BB, column 1. That is, in the cell i,j of MM we have the sum of the element-wise multiplication of all the numbers in the i -th row in AA and the j -th column in BB.

The following figure intuitively explains this idea:



The first thing to do is to determine the x and y axis index (i.e. row and column numbers):

```
__global__ void multiplication (float *A, float* B, float *C, int N)
```

```
{
```

```
    int ROW = blockIdx.y*blockDim.y+threadIdx.y;
```

```
    int COL = blockIdx.x*blockDim.x+threadIdx.x;
```

Then, we check that the row and column total does not exceed the number of actual rows and columns in the matrices. As the threads will access the memory in random order, we have to do this for preventing unnecessary threads from performing operations on our matrices. That is, we are making blocks and grids of a certain size, so that if we don't set our N as a multiple of that size, we would have more threads than we need:

```
    if (ROW < N && COL < N) {
```

We will not have any problem here as we are using square matrices, but it's always a good idea to keep this in mind. We then have to initialize the temporary variable tmp_sum for summing all the cells in the selected row and column. It is always a good procedure to specify the decimal points. So, let's write:

```
        float tmp_sum = 0.0f;
```

Now the straight forward part: as for the CPU code, we can use a “for” loop for computing the sum and then store it in the corresponding C cell.

```
        for (int i = 0; i < N; i++)
```

```
        {
```

```
            tmpSum += A[ROW * N + i] * B[i * N + COL];
```

```
        }
```

```
C[ROW * N + COL] = tmpSum;
```

The main code starts by declaring variables in this way:

```
int main()
{
    // declare arrays and variables
    int N = 16;
    int SIZE = N*N;
    vector<float> h_A(SIZE);
    vector<float> h_B(SIZE);
    vector<float> h_C(SIZE);
}
```

Then you also have to give values to the matrix for this there are a lot of method like making functions for manual input or using random numbers.

You also have to allocate memory as in the previous lab.

Then you also have to define the blocks and grids and you can also be able to use dim3 for this like

```
dim3 dimblock (BLOCK_SIZE, BLOCK_SIZE)
dim3 dimgrid (ceil(N/BLOCK_SIZE), ceil(N/BLOCK_SIZE))
```

Then at the end you have to free the memory like

```
cudaFree(d_Pin);
cudaFree(d_Pin1);
cudaFree(d_Pout);
```

Sequential C Code mxm.c

```
#include <stdio.h>

const int size = 100;

int main()
{
    int c, d, k, sum = 0;
    int first[size][size], second[size][size], multiply[size][size];

    // random elements of first matrix\n
```

```

for ( c = 0 ; c < size ; c++ )
for ( d = 0 ; d < size ; d++ )
first[c][d] = c + d;

// random elements of second matrix

for ( c = 0 ; c < size ; c++ )
for ( d = 0 ; d < size ; d++ )
second[c][d] = c * c ;

// multiplication matrix initialization with zero

for ( c = 0 ; c < size ; c++ )
for ( d = 0 ; d < size ; d++ )
multiply[c][d] = 0;

for ( c = 0 ; c < size ; c++ )
{
for ( d = 0 ; d < size ; d++ )
{
for ( k = 0 ; k < size ; k++ )
{
sum = sum + first[c][k]*second[k][d];
}
multiply[c][d] = sum;
sum = 0;
}
}

printf("Product of entered matrices:-\n");

for ( c = 0 ; c < size ; c++ )
{
for ( d = 0 ; d < size ; d++ )
printf("%d\t", multiply[c][d]);

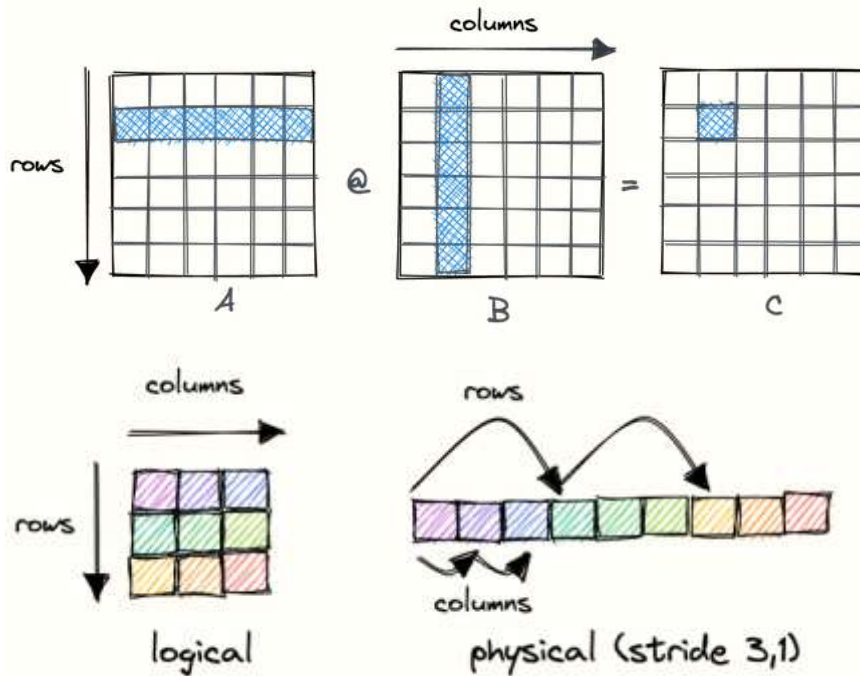
printf("\n");
}

return 0;
}

```

Lab Task:

- Write a code for Matrix Multiplication in CUDA C.



- Write a function for matrix multiplication to execute on a CPU. Find and print the execution time of both Cuda kernel and CPU function using timer library. Compare the results.
- Write a function to compute transpose of a matrix on both, CUDA and CPU. Find and print the execution of both kernel and CPU using timer library. Compare their results also.